

Student 15 **JERLIN J** 192312313RA

4 / 20 submitted

## Q1. Arrays

### CODE

```
#include<stdio.h>
int main()
{
    int production[12];
    int i;
    int total;
    float average;
    const int target = 5000;
    for(i=0;i<12;i++)
    {
        scanf("%d",&production[i]);
        if(production[i] == -1)
        {
            printf("Plant shut down due to equipment failure\n");
            break;
        }
        if(production[i]<0)
        {
            printf("Invalid production for month %d skipped\n",i+1);
            continue;
        }
        total += production[i];
        printf("Month %d ",i+1);
        if(production[i]>target)
            printf("10%% Performance Incentive\n");
        else if(production[i]<0.8*target)
            printf("Inspection Required\n");
        else
            printf("Target Achieved\n");
    }
    if(i>0)
        average = (float)total /i;
    else
        average = 0;
    printf("\nAnnual Productionn = %d\n",total);
    printf("Average Monthly Production = %2f\n",average);
    printf("Production Efficiency = %2f\n",(average/target)*100);
    return 0;
}
```

### INPUT

5200  
4800  
3900  
5100  
5000  
5300  
4500  
6000  
4900  
5200  
5100  
4800

### OUTPUT

(no output)

## Q2. Arrays

## CODE

```
#include<stdio.h>
int main()
{
    int quality[20],delay[20];
    int i,count=0;
    float sumQuality = 0 ,sumDelay = 0;
    for(i=0;i<20;i++)
    {
        scanf("%d %d",&quality[i],&delay[i]);
        if(quality[i]== -1)
            break;
        if(quality[i] == -2)
            continue;
        sumQuality += quality[i];
        sumDelay += delay[i];
        count++;
        if(quality[i]≥90 && delay[i]≤2)
            printf("Excellent\n");
        else if(quality[i]≥75 && quality[i]≤89 && delay[i]≤5)
            printf("Good\n");
        else if(quality[i]≥60 && quality[i]≤74)
            printf("Average");
        else
            printf("Needs Improvement\n");
    }
    if(count>0)
    {
        printf("Average Quality Score = %2f\n",sumQuality/count);
        printf("Average Delivery Delay = %2f\n",sumDelay/count);
    }
    return 0;
}
```

## INPUT

```
95 2
80 4
65 6
50 8
92 1
76 5
70 7
58 9
88 3
91 2
-1 0
```

## OUTPUT

(no output)

## Q6. Arrays

CODE

```
#include<stdio.h>
int main()
{
    int defects[30],accepted[30];
    int i,count = 0;
    float defectPercent , totalDefect =0, totalAccepted=0;
    for(i=0;i<30;i++)
    {
        scanf("%d",&defects[i]);
        if(defects[i] == -1)
            break;
        if(defects[i] == -2)
            continue;
        defectPercent = (defects[i]*100.0)/10000;
        accepted[i]=10000-defects[i];
        totalDefect += defectPercent;
        totalAccepted += accepted[i];
        count++;
        if(defectPercent <1)
            printf("Grade A\n");
        else if(defectPercent≤3)
            printf("Grade B\n");
        else if(defectPercent≤5)
            printf("Grade C\n");
        else
            printf("Reject Batch\n");
    }
    printf("Average Defect Percentage = %2f\n",totalDefect/count);
    printf("Average Accepted Chips = %2f\n",totalAccepted/count);
    return 0;
}
```

INPUT

```
50
120
350
600
-1
```

OUTPUT

(no output)

### Q7. Arrays

#### CODE

```
#include<stdio.h>
int main()
{
    int plan[20],used[20],capacity,remain,extra,i;
    for(i=0;i<20;i++)
    {
        scanf("%d %d",&plan[i],&used[i]);
        if(plan[i] == -1)
            break;
        if(plan[i] == -2)
            continue;
        if(plan[i]==1)
            capacity = 100;
        else if(plan[i]==2)
            capacity = 500;
        else
            capacity = 2000;
        remain = capacity - used[i];
        if(used[i]>capacity)
            extra = (used[i]-capacity)*4;
        else
            extra = 0;
        printf("Remaining Storage = %d GB\n",remain);
        if(extra>0)
            printf("Extra charge = RS.%d\n",extra);
        else
            printf("Extra Charge = Rs.0\n");
        if((used[i]*100)/capacity>95)
            printf("Warning\n");
        else
            printf("Storage OK\n");
    }
    return 0;
}
```

#### INPUT

```
1 80
2 490
3 2100
1 98
-1 0
```

#### OUTPUT

(no output)